Here we use a `for` loop to iterate over each byte in the `$str` buffer. `$str[0]` accesses the zeroth byte (`P`), `$str[1]` the next (`H`), and so on. The reason this works is because strings actually *are* arrays (in a sense). Strings in C (the language in which the PHP interpreter is written) are implemented as arrays of single bytes (the *char* C data type, which is something of a misnomer because it covers ASCII non-character byte values, as well as the 8-bit range *0x80–0xFF*), and PHP internally stores strings using a similar construct. Writing and maintaining code like this is perfectly fine but will quickly grow tiresome and is also very likely to be unnecessary because PHP has many (oh so many) use cases already covered.

Equal, Greater, or Less Than?

Table 8-8. *String comparison functions*

`strcmp()` `strnatcmp()`	Test if a string is less than, greater than, or equal to another
`strcasecmp()` `strnatcasecmp()`	Test if a string is less than, greater than, or equal to another, ignoring case for ASCII letters
`strncmp()` `strncasecmp()`	Binary-safe comparison of first N bytes
`substr_compare()`	Search sub-string for a matching string, restricted with offset and length
`strcoll()`	Locale-based comparison

Testing if one string equals another is a basic task, for which we can of course use logical == or ===. A closely related logical use case would be when sorting the order of strings, where it is necessary to test which value is greater (and the rules for establishing what counts as greater-than or lesser-than will vary; we will get to this a little later on in this section). You can use the spaceship operator for such a test: the expression `"a" <=> "a"` will return `0` (both are equal), `"ab" <=> "ac` returns `-1` (the left-hand operand is less than), `"a" <=> "B"` returns `1` (the right-hand operand is less than). The core string functions listed in the table above can also be used for testing strings in this way.

`strcmp(), strcasecmp(), strnatcmp(), strnatcasecmp()`